

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: РАЗРАБОТКА ВИЗУАЛИЗАТОРА АЛГОРИТМА ПРИМА
НА ЯЗЫКЕ JAVA**

Студент гр. 4344

К.А. Рогачевский

Студент гр. 4344

М.И. Захарова

Студент гр. 4381

А.И. Кяримов

Руководитель

Р.П. Шестопапов

Санкт-Петербург
2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: К.А. Рогачевский Группа: 4344

Студент: М.И. Захарова Группа: 4344

Студент: А.И. Кяримов Группа: 4381

Тема практики: Разработка визуализатора алгоритма Прима на языке Java

Задание на практику:

Командная итеративная разработка визуализатора алгоритма на Java с графическим интерфейсом.

Алгоритм: алгоритм Прима.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 15.07.2026

Дата защиты отчета: 17.07.2026

Студент	К.А. Рогачевский
Студент	М.И. Захарова
Студент	А.И. Кяримов
Руководитель	Р.П. Шестопалов

АННОТАЦИЯ

Данный отчет описывает процесс командной разработки программного приложения — визуализатора алгоритма Прима. В ходе работы был создан графический интерфейс на языке Java с использованием библиотек Swing и AWT. Реализованы функциональности для ручного построения графа, загрузки и сохранения данных из/в файл, а также пошаговая и автоматическая визуализация работы алгоритма. Применялись принципы итеративной разработки и распределения ролей в команде. В результате разработано приложение, позволяющее наглядно демонстрировать процесс поиска минимального остовного дерева во взвешенном графе.

SUMMARY

This report describes the process of team development of a software application — a visualizer for Prim's algorithm. During the work, a graphical interface was created in Java using Swing and AWT libraries. Functionalities for manual graph construction, loading and saving data from/to files, as well as step-by-step and automatic visualization of the algorithm were implemented. Principles of iterative development and role distribution within the team were applied. As a result, an application was developed that allows for a clear demonstration of the process of finding the minimum spanning tree in a weighted graph.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	5
1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Предметная область.....	6
1.2 Задачи практики.....	6
2 РЕЗУЛЬТАТЫ РАБОТЫ.....	7
2.1. Вводное задание: изучение языка Java.....	7
2.2. Составление спецификации программы и плана разработки.....	8
2.2.1 Исходные требования к программе.....	17
2.2.1.1 Требования к вводу исходных данных.....	18
2.2.1.2 Требования к визуализации.....	18
2.2.1.3 Требования к управлению.....	19
2.2.2 План разработки и распределение ролей в бригаде.....	20
2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом.....	21
2.3.1. Структуры данных.....	21
2.3.2. Основные методы.....	21
2.3.3. Тестирование.....	22
2.3.3.1. План тестирования.....	22
2.3.3.2. Тестирование графического интерфейса.....	22
2.3.3.3. Тестирование алгоритма.....	27
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ.....	30
4 ОТЗЫВ РУКОВОДИТЕЛЯ.....	31
ЗАКЛЮЧЕНИЕ.....	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	33
ПРИЛОЖЕНИЕ А.....	34

ВВЕДЕНИЕ

В современном мире задачи оптимизации на графах играют ключевую роль в различных областях, от проектирования телекоммуникационных сетей и транспортных маршрутов до биоинформатики и машинного обучения. Одной из фундаментальных задач теории графов является поиск минимального остовного дерева (Minimum Spanning Tree, MST), которое соединяет все вершины графа с минимально возможной суммарной стоимостью ребер.

Алгоритм Прима — это классический жадный алгоритм, решающий данную задачу [1]. Визуализация его работы является мощным инструментом для образования и научного познания, так как позволяет наглядно продемонстрировать пошаговую логику процесса, что значительно упрощает понимание алгоритма по сравнению с его абстрактным текстовым или псевдокодовым описанием.

Целью учебной практики являлась разработка программного приложения "Визуализатор алгоритма Прима" на языке Java, предоставляющего пользователю интерактивный интерфейс для построения графов и пошагового наблюдения за работой алгоритма.

Для достижения поставленной цели решались следующие задачи:

Изучение основ объектно-ориентированного программирования на языке Java и библиотек для создания графического интерфейса (Swing, AWT).

Командная разработка спецификации и плана работы над проектом.

Реализация графического интерфейса пользователя, позволяющего взаимодействовать с графом

Реализация логики алгоритма Прима и его интеграция с визуальным представлением.

Создание системы управления пошаговым выполнением алгоритма и отображением текущего состояния.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Предметная область работы — это теория графов и разработка программного обеспечения с графическим интерфейсом. Алгоритмы поиска MST широко применяются на практике: в проектировании дорожных сетей, для прокладки кабелей связи, в кластерном анализе и многих других задачах, где требуется соединить объекты с наименьшими затратами.

Разработанный визуализатор может быть использован:

В образовательном процессе (лекции, лабораторные работы) для наглядной демонстрации принципов работы алгоритма Прима.

В качестве прототипа для более сложных систем анализа графов.

1.2 Задачи практики

В рамках практики перед командой были поставлены следующие задачи:

Вводное задание: Изучить основы языка Java, синтаксис, принципы ООП и стандартные библиотеки для создания графических интерфейсов (Swing).

Составление спецификации: Разработать подробную спецификацию программы, включающую назначение, функциональные требования, описание пользовательского интерфейса и форматов данных. Согласовать с руководителем план разработки и распределить роли в команде.

Командная итеративная разработка: Реализовать программный продукт в соответствии со спецификацией и планом. Включает в себя реализацию всех функций ввода/вывода, визуализацию алгоритма, управление процессом выполнения и тестирование.

Оформление отчета: Подготовить итоговый отчет о проделанной работе в соответствии с установленными требованиями.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание: изучение языка Java

В рамках вводного задания был пройден онлайн-курс «Java. Базовый курс» на образовательной платформе Stepik [2]. Данный курс был выбран как наиболее полно охватывающий фундаментальные основы языка, необходимые для разработки графического приложения с нуля.

В ходе изучения курса были освоены следующие темы:

Основы языка Java:

Синтаксис языка, переменные, операторы, условные конструкции и циклы.

Принципы объектно-ориентированного программирования: наследование, полиморфизм, инкапсуляция.

Работа с коллекциями (ArrayList, HashMap).

Обработка ошибок и исключения:

Понятие исключительной ситуации и исключения в Java.

Иерархия классов исключений (Throwable, Exception, Error, RuntimeException).

Обработка исключений с помощью конструкций try-catch-finally и try-with-resources.

Проброс исключений с использованием ключевого слова throws.

Создание собственных классов-исключений.

Для закрепления материала были выполнены практические задания.

Самостоятельное изучение дополнительных тем:

Помимо материалов базового курса, для успешной реализации поставленной задачи потребовалось самостоятельное изучение ряда тем, которые не были в полной мере освещены в рамках онлайн-курса:

В рамках курса на Stepik основное внимание уделялось разработке приложений без графического интерфейса, поэтому освоение графических компонентов (JFrame, JPanel, JButton), менеджеров компоновки (BorderLayout, FlowLayout), обработчиков событий мыши и клавиатуры

проводилось по официальной документации Oracle и специализированным руководствам.

Работа с графикой и отрисовка на холсте. Для реализации визуализации графа были изучены возможности класса Graphics и его методов (drawOval, drawLine, drawString) для отрисовки вершин и рёбер, а также механизм перерисовки компонентов через метод paintComponent() и вызов repaint().

Многопоточность и асинхронное выполнение. Для реализации автоматического пошагового выполнения алгоритма с регулируемой скоростью были изучены классы Timer и SwingWorker, а также принципы работы с потоками в контексте GUI-приложений (правила Event Dispatch Thread).

Механизм перетаскивания (Drag-and-Drop). Для реализации загрузки файлов перетаскиванием в окно приложения были изучены классы DropTarget и DropTargetAdapter, позволяющие обрабатывать события перетаскивания файлов из операционной системы.

Работа с диалоговыми окнами. Самостоятельно были изучены классы JFileChooser для выбора файлов и JOptionPane для создания диалоговых окон ввода данных и отображения сообщений.

Таким образом, сочетание прохождения структурированного онлайн-курса с самостоятельным изучением узкоспециализированных тем позволило получить все необходимые знания для реализации полноценного графического приложения.

2.2. Составление спецификации программы и плана разработки

Спецификация программы «Визуализатор алгоритма Прима»

Назначение программы:

Приложение предназначено для наглядной демонстрации работы алгоритма Прима поиска минимального остовного дерева (MST) во взвешенном неориентированном графе. Программа визуализирует граф,

позволяет пользователю задавать его структуру и пошагово наблюдать за процессом построения дерева.

Функциональные возможности:

Программа предоставляет пользователю следующие возможности:

Работа с графом:

Добавление вершины на холст по правому клику мыши. Новая вершина появляется в месте клика и получает автоматическое уникальное имя.

Удаление вершины по правому клику на существующей вершине. При удалении вершины все инцидентные ей рёбра также удаляются.

Добавление ребра путём перетаскивания левой кнопкой мыши от вершины к вершине. После отпускания открывается диалоговое окно для ввода веса ребра (целое положительное число).

Очистка графа кнопкой «Очистить» — удаление всех вершин и рёбер с холста.

Загрузка графа из текстового файла через диалоговое окно выбора файла.

Загрузка графа через диалоговое окно ручного ввода (кнопка «Загрузить») в формате, описанном в разделе 1.5.

Загрузка графа перетаскиванием текстового файла в окно приложения (Drag-n-Drop).

Сохранение текущего графа в файл.

Управление алгоритмом:

Запуск алгоритма Прима кнопкой «Старт».

Пошаговое выполнение алгоритма кнопкой «Шаг» — каждое нажатие выполняет один шаг.

Автоматическое пошаговое выполнение с регулируемой скоростью (слайдер в нижней панели).

Пауза — остановка автоматического выполнения.

Шаг назад (кнопка «Назад») — откат на одно действие.

Сброс алгоритма к начальному состоянию (кнопка «Рестарт») без удаления графа.

Визуализация и информация:

Цветовая подсветка вершин и рёбер в процессе работы алгоритма (цветовая схема описана в разделе 1.4).

Текстовые пояснения каждого шага в строке состояния.

Справочное окно с описанием всех функций (кнопка «Помощь»).

Информация о разработчиках с ФИО, группой и фотографией (кнопка в нижней панели).

Обработка всех исключительных ситуаций с выводом понятных сообщений пользователю.

Ползунки прокрутки для работы с большими графами.

Пользовательский интерфейс (рис. 1):

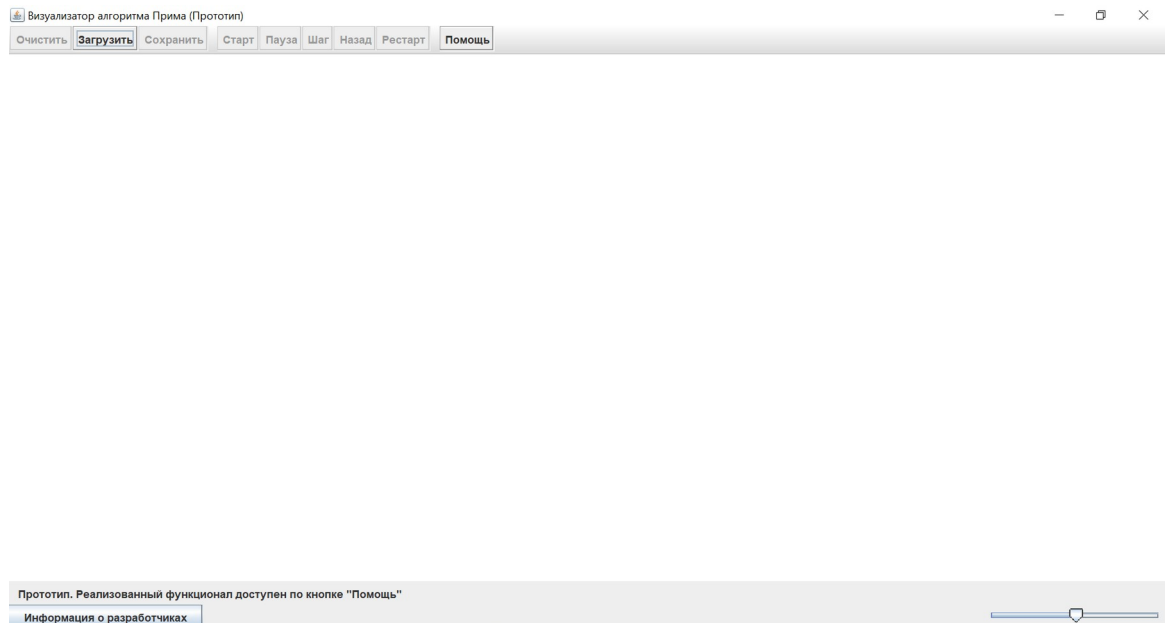


Рисунок 1 – Прототип интерфейса

Холст является центральной областью для рисования графа. Вершины отображаются в виде кругов синего цвета с названиями. Рёбра отображаются в виде линий с указанием веса рядом. Взаимодействие с холстом осуществляется с помощью мыши:

Правый клик по пустому полю — добавление вершины.

Правый клик по вершине — удаление вершины и всех связанных с ней рёбер.

Левая кнопка мыши от вершины к вершине — добавление ребра с запросом веса.

Панель инструментов расположена в верхней части окна и содержит следующие кнопки:

«Очистить» — удаление всех вершин и рёбер с холста.

«Загрузить» — открытие диалогового окна для ручного ввода графа.

«Сохранить» — сохранение текущего графа в файл.

«Старт» — запуск алгоритма Прима.

«Пауза» — остановка автозапуска.

«Шаг» — переход к следующему шагу алгоритма.

«Назад» — возврат на один шаг назад.

«Рестарт» — сброс алгоритма к начальному состоянию.

«Помощь» — открытие справочного окна с описанием всех функций.

Строка состояния расположена в нижней части окна и содержит:

Текстовое поле с пояснением текущего шага или выполненного действия.

Слайдер регулировки скорости автозапуска.

Кнопку «Информация о разработчиках», открывающую диалоговое окно с ФИО, группой и фотографией команды.

Поле ручного ввода графа:

Для удобства тестирования и быстрого ввода данных в интерфейсе предусмотрено диалоговое окно, вызываемое кнопкой «Загрузить», с возможностью ввода графа в текстовом формате.

Формат ввода:

Первая строка — количество вершин

Последующие строки — рёбра в формате: <вершина1> <вершина2>
<вес>

Пример ввода:

4

A B 1

A C 4

B C 2

B D 5

Данная функция позволяет пользователю проверить работу алгоритма без необходимости создания отдельного файла, что особенно полезно на этапе разработки и тестирования.

Визуализация алгоритма (пошагово):

Цветовая схема:

Синий (DEFAULT): Вершины и рёбра, ещё не обработанные.

Зелёный (IN_MST): Вершины и рёбра, уже добавленные в минимальное остовное дерево.

Красный (ACTIVE): Текущая обрабатываемая вершина.

Голубой (CONSIDERED): Текущее рассматриваемое ребро (кандидат на добавление).

Пример пошагового процесса:

Шаг	Действие	Текстовое пояснение
0	Выбор стартовой вершины (по умолчанию — первая)	«Начало работы. Стартовая вершина: А»
1	Поиск минимального ребра из дерева	«Рассматриваем ребро А-В (вес 1) — минимальное»
2	Добавление вершины и ребра в дерево	«Добавляем вершину В и ребро А-В в дерево»

3	Обновление множества доступных рёбер	«Теперь доступны рёбра: A-C (4), B-C (2), B-D (5)»
...	...	...
N	Все вершины добавлены	«Минимальное остовное дерево построено! Суммарный вес: 12»

Формат входных и выходных данных:

Входной файл (input.txt):

4

A B 1

A C 4

B C 2

B D 5

Первая строка — количество вершин.

Последующие строки — рёбра в формате: <вершина1> <вершина2>
<вес>.

Веса — целые положительные числа.

Вершины могут быть обозначены буквами или цифрами.

Выходной файл (result.txt):

Минимальное остовное дерево:

A - B (1)

B - C (2)

B - D (5)

Суммарный вес: 8

Или в том же формате, что и входной, но только с рёбрами дерева.

Ручной ввод:

Ручной ввод графа осуществляется через диалоговое окно, вызываемое кнопкой «Загрузить». Формат ввода полностью соответствует формату входного файла.

План разработки

Этапы и распределение задач:

№	Задача	Исполнитель	Срок	Ожидаемый результат
0	Прототип	Все	06.07 –10.07	Графический интерфейс без логики
0. 1	Создать спецификацию и план работы	Марина	06.07	Конкретика
0. 2	Создать структуру проекта и репозиторий	Амирхан, Кирилл	06.07 -07.07	Каркас приложения
0. 3	Реализовать классы Vertex, Edge, Graph	Амирхан	07.07 -08.07	Модель данных
0. 4	Создать главное окно с холстом, кнопками, строкой состояния и полем для ручного ввода графа	Кирилл	07.07 -08.07	GUI

0.5	Реализовать добавление вершины по клику мыши	Кирилл	07.07 -08.07	Взаимодействие
0.6	Подготовить исполнимый .jar и сдать прототип	Марина	09.07	Готово к показу
1	Альфа-версия	Все	10.07 -13.07	Загрузка графа + финальный результат
1.1	Реализовать парсер входного файла	Марина	11.07	Граф загружается
1.2	Отрисовка загруженного графа на холсте (подписи вершин и рёбер)	Кирилл	11.07	Граф отображается
1.3	Реализовать алгоритм Прима (вычисление MST)	Амирхан	11.07	Алгоритм корректен
1.4	Подсветка финального результата (рёбра дерева — зелёные)	Кирилл	12.07	Визуализация результата
1.	Сохранение	Амирхан	12.07	Экспорт

5	е результата в файл	н		работает
1. 6	Собрать .jar и сдать альфа- версию	Марина	13.07	Готово к показу
2	Бета- версия	Все	13.07 –15.07	Пошаговая визуализация
2. 1	Реализовать пошаговое выполнение алгоритма	Амирхан	14.07	Шаг за шагом
2. 2	Добавить цветовую подсветку текущего шага	Кирилл	14.07	Визуализация активна
2. 3	Реализовать кнопки «Шаг вперёд», «Автозапуск», «Сброс»	Кирилл	14.07	Управление
2. 4	Добавить текстовые пояснения в строку состояния	Марина	15.07	Информативност ь
2. 5	Собрать .jar и сдать бета- версию	Марина	15.07	Готово к показу
3	Финальная	Все	15.07	Все функции +

	версия (релиз)		–17.07	полировка
3.1	Реализовать «Шаг назад» (стек состояний)	Амирхан	16.07	Откат работает
3.2	Добавить обработку исключений (некорректные файлы, отсутствие файлов)	Амирхан	16.07	Стабильность
3.3	Реализовать ползунки прокрутки для больших графов	Кирилл	16.07	Масштабируемость
3.4	Доработать кнопку «О разработчиках»	Кирилл	16.07	Информация о команде
3.5	Финальное тестирование, исправление багов	Марина	16.07	Качество
3.6	Собрать финальный .jar, подготовить отчёт	Марина	16.07	Готово к сдаче

2.2.1 Исходные требования к программе

В ходе совместного обсуждения и согласования с руководителем были определены следующие требования к разрабатываемому приложению.

2.2.1.1 Требования к вводу исходных данных

Программа должна поддерживать следующие способы ввода графа:

1. Интерактивное построение с помощью мыши на холсте (реализовано в классе `MouseController`):
 - Добавление вершины по правому клику на пустом месте холста. Новая вершина должна получать автоматическое уникальное имя (реализовано в методе `placeVertex()`).
 - Удаление вершины по правому клику на существующей вершине. При удалении вершины все инцидентные ей рёбра должны удаляться автоматически (реализовано в методе `removeVertex()` класса `Graph`).
 - Добавление ребра перетаскиванием левой кнопкой мыши от вершины к вершине. После отпускания кнопки должно открываться диалоговое окно для ввода веса ребра (реализовано в методе `mouseReleased()` класса `MouseController` с использованием `JOptionPane`).
2. Загрузка из текстового файла через диалоговое окно выбора файла (кнопка «Загрузить» в классе `Toolbar`).
3. Загрузка через ручной ввод в специальном диалоговом окне, которое должно открываться по нажатию кнопки «Загрузить» (реализовано с использованием `JOptionPane.showInputDialog()`).
4. Загрузка перетаскиванием (Drag-and-Drop) текстового файла в окно приложения (реализовано в классе `DragnDropHandler`, наследующем `TransferHandler`).

2.2.1.2 Требования к визуализации

Программа должна обеспечивать наглядное отображение графа и процесса работы алгоритма:

Отображение графа (реализовано в методе `paintComponent()` класса `Canvas`):

- Вершины должны отображаться в виде кругов на холсте.
- Каждая вершина должна иметь подпись с её уникальным именем.
- Рёбра должны отображаться в виде линий, соединяющих вершины.
- Рядом с каждым ребром должен отображаться его вес.

Текстовое сопровождение:

В строке состояния должна отображаться текстовая информация о текущем шаге алгоритма (реализовано через `statusLabel` в классе `Application` и метод `setStatus()`).

Масштабирование:

- Для работы с большими графами должны быть предусмотрены ползунки прокрутки.

2.2.1.3 Требования к управлению

Пользователю должны быть доступны следующие элементы управления (все кнопки реализованы в классе `Toolbar`):

Управление графом:

- «Очистить» — удаление всех вершин и рёбер с холста.
- «Сохранить» — сохранение текущего графа в файл.

Управление алгоритмом:

- «Старт» — запуск алгоритма Прима.
- «Пауза» — остановка автоматического выполнения.
- «Шаг» — переход к следующему шагу алгоритма (пошаговое выполнение).
- «Назад» — возврат на один шаг назад (откат состояния).
- «Рестарт» — сброс алгоритма к начальному состоянию без удаления графа (реализован через метод `reset()` класса `Graph`).

Настройка автоматического выполнения:

Регулировка скорости автоматического пошагового выполнения с помощью слайдера (реализован в классе BottomPanel).

Справочная информация:

- «Помощь» — открытие справочного окна с описанием всех функций (реализовано в классе Toolbar).
- Кнопка с информацией о разработчиках (реализована в классе BottomPanel с использованием JOptionPane).

Обработка ошибок:

Все исключительные ситуации должны обрабатываться с выводом понятных сообщений пользователю (реализовано с использованием блоков try-catch в классах MouseController, DragDropHandler и Toolbar).

2.2.2 План разработки и распределение ролей в бригаде

В команде были распределены следующие роли и определены этапы разработки:

- Марина Захарова (гр. 4344): Менеджер проекта, тестирование, документация. Отвечала за составление спецификации, плана работы, написание итогового отчета и финальное тестирование.
- Кирилл Рогачевский (гр. 4344): Разработчик интерфейса. Отвечал за создание GUI компонентов, обработку событий мыши и отрисовку графа на холсте.
- Амирхан Кяримов (гр. 4381): Разработчик бэкенда. Отвечал за реализацию структур данных (граф, вершины, ребра), логику алгоритма Прима, его пошаговое выполнение и управление состояниями.

План разработки:

1. Прототип (06.07 – 10.07): Создание каркаса приложения с основными элементами интерфейса и базовым взаимодействием с мышью.
2. Альфа-версия (10.07 – 13.07): Реализация загрузки и сохранения графа, а также отображение финального результата работы алгоритма.

3. Бета-версия (13.07 – 15.07): Реализация полноценной пошаговой визуализации алгоритма с цветовой подсветкой и текстовыми пояснениями.

4. Релиз (15.07 – 17.07): Полировка интерфейса, добавление дополнительных функций (кнопка "Назад", обработка ошибок, информация о разработчиках), окончательное тестирование и подготовка отчета.

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

Процесс разработки в целом соответствовал первоначальному плану. Итеративный подход позволил своевременно выявлять и исправлять ошибки, а также гибко реагировать на уточнения требований. Коммуникация между членами команды осуществлялась через систему контроля версий Git и ежедневные собрания.

2.3.1. Структуры данных

В основе приложения лежит модель данных, представленная классами:

- Vertex: хранит уникальное имя (String) и координаты (int x, int y) для отображения на холсте.
- Edge: хранит ссылки на две вершины (Vertex source, Vertex target) и вес ребра (int weight).
- Graph: содержит коллекции вершин (ArrayList<Vertex>) и ребер (ArrayList<Edge>). Предоставляет методы для добавления, удаления и поиска элементов.
- StepState: вспомогательный класс для реализации функции "Шаг назад". Хранит состояние графа (список ребер в MST, подсвеченные элементы) на каждом шаге.

2.3.2. Основные методы

Основные методы (функциональные модули):

- GraphParser: Статический класс с методом parseGraph(File file), который читает файл и создает объект Graph. Включает в себя проверку корректности ввода (валидацию).

- **PrimAlgorithm:** Класс, содержащий логику алгоритма. Метод `runStep()` выполняет один шаг алгоритма, обновляя внутреннее состояние и возвращая информацию о текущем действии (добавленное ребро, рассматриваемое ребро и т.д.). Метод `reset()` возвращает алгоритм в начальное состояние.
- **GraphPanel:** Класс, наследующий `JPanel`, отвечает за отрисовку всех компонентов графа. Он содержит переопределенный метод `paintComponent(Graphics g)`, который в зависимости от текущего состояния алгоритма раскрашивает вершины и ребра в соответствующие цвета.

2.3.3. Тестирование

Тестирование разработанного приложения проводилось на всех этапах разработки и было направлено на проверку корректности работы пользовательского интерфейса, логики алгоритма и обработки исключительных ситуаций.

2.3.3.1. План тестирования

Тестирование проводилось по следующему плану:

1. Модульное тестирование: Проверка корректности работы отдельных классов и методов (структуры данных, парсер файлов, алгоритм Прима).
2. Интеграционное тестирование: Проверка взаимодействия между компонентами системы (графический интерфейс с бэкендом).
3. Системное тестирование: Проверка работы приложения в целом в соответствии с требованиями спецификации.
4. Приемочное тестирование: Демонстрация работы приложения руководителю практики и получение обратной связи.
5. Тестирование пользовательского интерфейса: Включает в себя проверку всех элементов управления, отклика на действия мыши и корректность отображения графической информации.

2.3.3.2. Тестирование графического интерфейса

Тестирование графического интерфейса проводилось вручную для проверки удобства и корректности взаимодействия пользователя с приложением.

1. Проверка функциональности добавления и удаления вершин и рёбер:

Действие: Правый клик на пустом месте холста.

Ожидаемый результат: Добавление новой вершины с уникальным именем.

Результат: Пройдено успешно.

Примечание: Имена присваиваются автоматически в алфавитном порядке (А, В, С...). Переименование вершин не предусмотрено спецификацией, но пользователь может удалить вершину и добавить её заново в нужном порядке.

Действие: Правый клик на существующей вершине.

Ожидаемый результат: Удаление вершины и всех инцидентных ей рёбер.

Результат: Пройдено успешно.

Действие: Перетаскивание левой кнопкой мыши от одной вершины к другой.

Ожидаемый результат: Появление диалогового окна для ввода веса ребра. После ввода веса на холсте появляется новое ребро с указанным весом.

Результат: Пройдено успешно.

Пример скриншота (рис. 2):

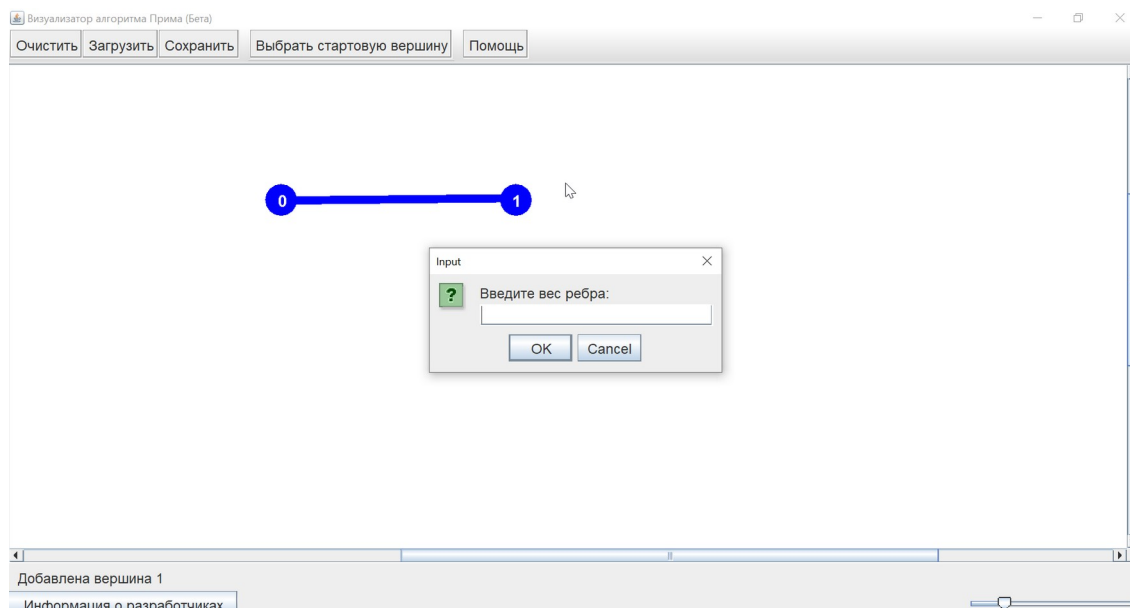


Рисунок 2 – Диалоговое окно для ввода веса

2. Проверка работы загрузки и сохранения графа:

Действие: Нажатие кнопки «Сохранить» и выбор места для сохранения файла.

Ожидаемый результат: Создание текстового файла с данными текущего графа в корректном формате.

Результат: Пройдено успешно.

Действие: Нажатие кнопки «Загрузить» и выбор текстового файла с графом.

Ожидаемый результат: Очистка холста и отрисовка графа из файла.

Результат: Пройдено успешно.

Пример скриншота (рис. 3):

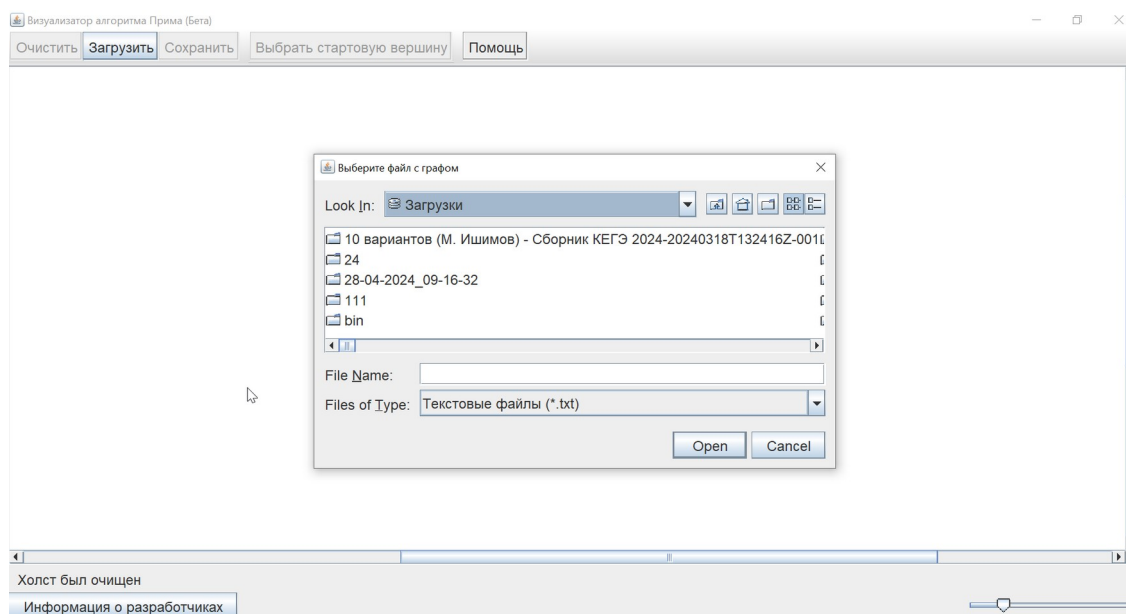


Рисунок 3 – Загрузка из файла

Действие: Нажатие кнопки «Загрузить» и ввод данных вручную в текстовом поле диалогового окна.

Ожидаемый результат: Отрисовка введенного графа.

Результат: Пройдено успешно.

Действие: Перетаскивание текстового файла в окно приложения.

Ожидаемый результат: Автоматическая загрузка графа из перетянутого файла.

Результат: Пройдено успешно.

3. Проверка управления алгоритмом:

Действие: Построение графа и нажатие кнопки «Старт».

Ожидаемый результат: Запуск алгоритма Прима с выбором стартовой вершины (первой по алфавиту).

Результат: Пройдено успешно.

Действие: Нажатие кнопки «Шаг».

Ожидаемый результат: Выполнение одного шага алгоритма, обновление цветовой подсветки и текстового пояснения в строке состояния.

Результат: Пройдено успешно.

Пример скриншота (рис. 4):

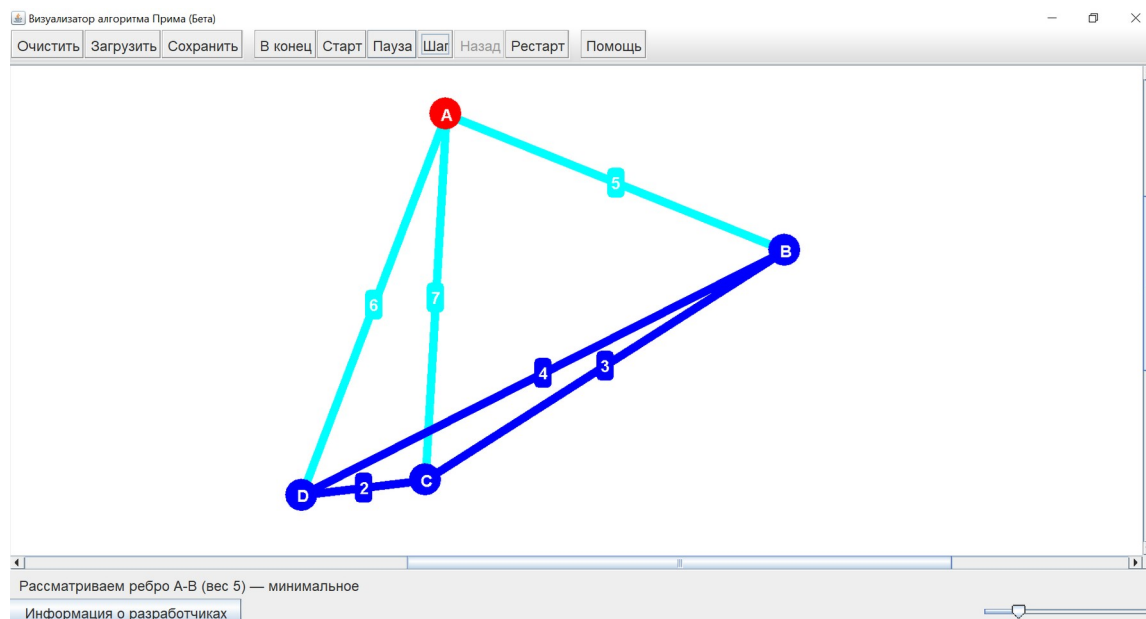


Рисунок 4 – Пошаговое выполнение алгоритма Прима

Действие: Включение автоматического выполнения и регулировка скорости слайдером.

Ожидаемый результат: Автоматическое выполнение алгоритма с заданной скоростью.

Результат: Пройдено успешно.

Действие: Нажатие кнопки «Назад» после выполнения нескольких шагов.

Ожидаемый результат: Откат состояния алгоритма на один шаг назад с соответствующим обновлением визуализации.

Результат: Пройдено успешно.

Действие: Нажатие кнопки «Рестарт».

Ожидаемый результат: Полный сброс алгоритма в начальное состояние. Граф остается на месте, все элементы становятся синими.

Результат: Пройдено успешно.

4. Проверка справочной информации:

Действие: Нажатие кнопки «Помощь».

Ожидаемый результат: Открытие окна с текстовым описанием всех функций программы.

Результат: Пройдено успешно.

Действие: Нажатие кнопки информации о разработчиках.

Ожидаемый результат: Открытие окна с ФИО, группой и фотографией команды.

Результат: Пройдено успешно.

5. Проверка масштабирования:

Действие: Создание большого количества вершин (за пределами видимой области).

Ожидаемый результат: Появление ползунков прокрутки для навигации по холсту.

Результат: Пройдено успешно.

2.3.3.3. Тестирование алгоритма

Для проверки корректности работы алгоритма Прима было выполнено несколько тестовых сценариев, включая проверку на графах с различной структурой и количеством вершин. Результаты работы алгоритма сравнивались с эталонными значениями минимального остовного дерева.

Тестовый пример №1: Простой граф

Исходный граф:

A

B

C

D

A B 1

A C 4

B C 2

B D 5

Ожидаемое остовное дерево (MST):

A - B (1)

B - C (2)

B - D (5)

Суммарный вес: 8

Результат: Пройдено успешно. Алгоритм выбрал именно эти ребра.

Тестовый пример №2: Граф с несколькими вершинами одинакового веса

Исходный граф:

A

B

C

D

A B 2

A C 2

B C 1

B D 3

C D 3

Ожидаемое остовное дерево (MST):

A - B (2) (или A-C (2))

B - C (1)

B - D (3) (или C-D (3))

Суммарный вес: 6

Важно отметить, что при наличии нескольких рёбер с одинаковым весом, алгоритм может выбрать любой из вариантов, сохраняя минимальность общего веса.

Результат: Пройдено успешно. Алгоритм корректно обработал ребра с одинаковым весом и построил остовное дерево с суммарным весом 6.

Пример скриншота:

(Вставьте сюда скриншот финального результата работы алгоритма для данного графа с подсвеченными рёбрами MST)

Тестовый пример №3: Граф с отрицательными весами (проверка на необработанное исключение)

Действие: Попытка ввести отрицательный вес ребра.

Ожидаемый результат: Валидация должна не допустить ввод отрицательного числа, отобразить сообщение об ошибке.

Результат: Пройдено успешно. В диалоговом окне ввода веса предусмотрена проверка; ввод нечислового или отрицательного значения приводит к выводу сообщения об ошибке, и ребро не создается.

Тестовый пример №4: Некорректный входной файл

Действие: Загрузка файла с неправильным форматом (например, отсутствие количества вершин в первой строке, неверное число рёбер, нечисловое значение веса).

Ожидаемый результат: Вывод понятного пользователю сообщения об ошибке с указанием причины.

Результат: Пройдено успешно. Все исключительные ситуации перехватываются и выводятся в диалоговом окне `JOptionPane.showMessageDialog()`.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе разработки были использованы следующие технологии и инструменты:

- Язык программирования Java (версия 21): Выбран как основной язык разработки благодаря своей кроссплатформенности, мощной стандартной библиотеке и широкому применению в корпоративной среде.
- Библиотеки Java Swing и AWT: Использованы для построения полноценного графического пользовательского интерфейса. JFrame, JPanel, JButton, JLabel и другие компоненты позволили создать функциональное и удобное окно приложения.
- Система контроля версий Git: Применялась для управления исходным кодом, отслеживания изменений и координации работы команды (работа в ветках, разрешение конфликтов). Репозиторий хранился на платформе GitHub [3].
- Среда разработки IntelliJ IDEA: Использовалась как основная IDE благодаря мощным инструментам рефакторинга, дебаггеру и удобной интеграции с Git.
- Java Collections Framework: Широко применялись стандартные коллекции (ArrayList, HashMap) для хранения вершин, ребер и управления состояниями алгоритма [4].

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на <ваша_оценка>.

ЗАКЛЮЧЕНИЕ

В результате прохождения учебной практики была успешно разработана программа "Визуализатор алгоритма Прима" на языке Java.

В рамках решения первой задачи были изучены основы языка Java, принципы ООП и библиотеки Swing, что позволило создать полноценное графическое приложение.

Вторая задача была решена путем разработки подробной спецификации и четкого плана работ, который позволил эффективно распределить роли в команде и организовать итеративный процесс разработки.

Основная третья задача (командная разработка) была выполнена в полном объеме. Приложение поддерживает все заявленные функции: ручное построение графа, загрузку из файла и по ручному вводу, сохранение, а главное — пошаговую и автоматическую визуализацию работы алгоритма Прима с цветовой подсветкой и текстовыми пояснениями. Были реализованы дополнительные функции, такие как откат на шаг назад и обработка ошибок ввода, что сделало программу еще более удобной.

Таким образом, все поставленные задачи практики были решены, цель достигнута. Разработанное приложение может быть использовано в образовательных целях для наглядной демонстрации работы алгоритма Прима.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ, 3-е издание. – М.: Вильямс, 2013. – 1328 с. (Глава 23. Минимальные остовные деревья).
2. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/Java-Базовый-курс-187/syllabus> (дата обращения: 06.07.2026).
3. The State of the Octoverse [Электронный ресурс]. URL: <https://octoverse.github.com> (дата обращения: 07.07.2026).
4. Седжвик, Р., Уэйн, К. Алгоритмы на Java, 4-е издание. – М.: Диалектика, 2015. – 848 с.

ПРИЛОЖЕНИЕ А

Представляется финальный вид пользовательского интерфейса приложения (рис. 5).

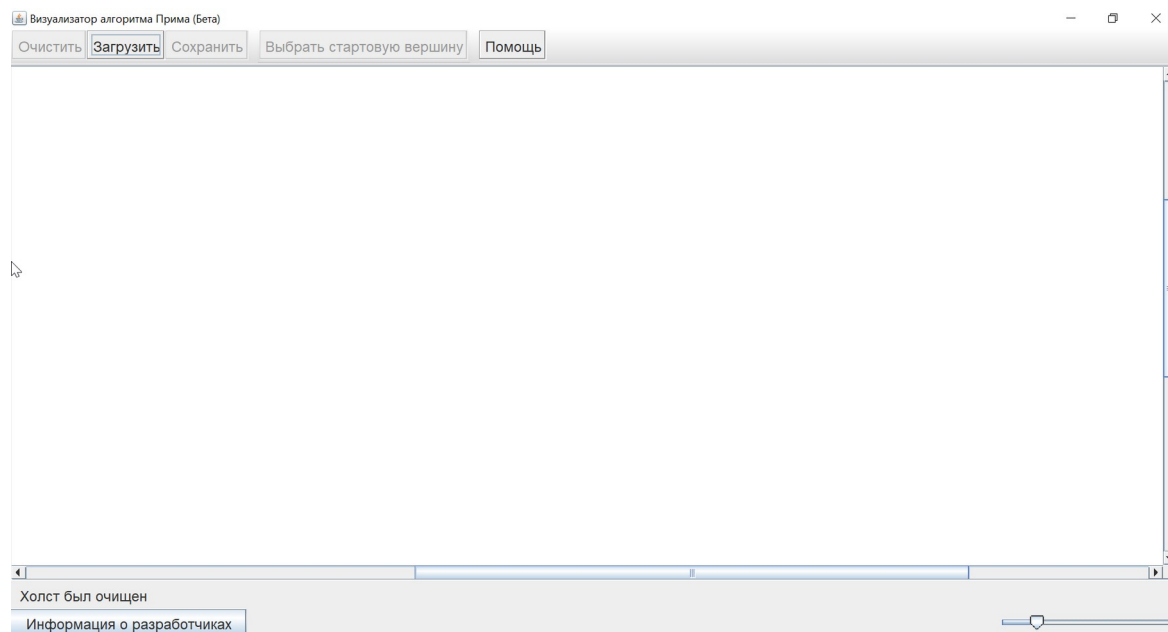


Рисунок 5 – Релизный вид продукта